

Iterative Statements

- **Introduction**
- **The `while` statement**
- **The `do/while` statement**
- **The `for` Statement**

Iterative Statement

- **Objective**
- **To understand iteration as means of controlling program flow**
- **To know the three forms of iterations:**
 - **while**
 - **do/while**
 - **for**

Iteration

- We have discussed two forms of Java statements – sequence and selection.
- There is a third type of Java statement – the iterative statements, commonly called loop.
- Iterative statements cause a certain statement or block of statements to be repeated.
- The statement or block of statements is referred to as the loop body.
- How does the program know to execute the loop body repeatedly?
- Answer - a conditional expression is used to determine this.

Iteration

- **Problem - add all the integers from 1 to 1000.**
- **Here is one approach**

$$1 + 1 = 2$$

$$2 + 1 = 3$$

$$3 + 1 = 4$$

$$4 + 1 = 5$$

$$5 + 1 = 6$$

:

:

:

Surely it will not be long before you realize that this approach is repetitive and laborious

Iteration

- Java provides three forms of iterative statements:
- *while*
- *do...while*, and
- *for*

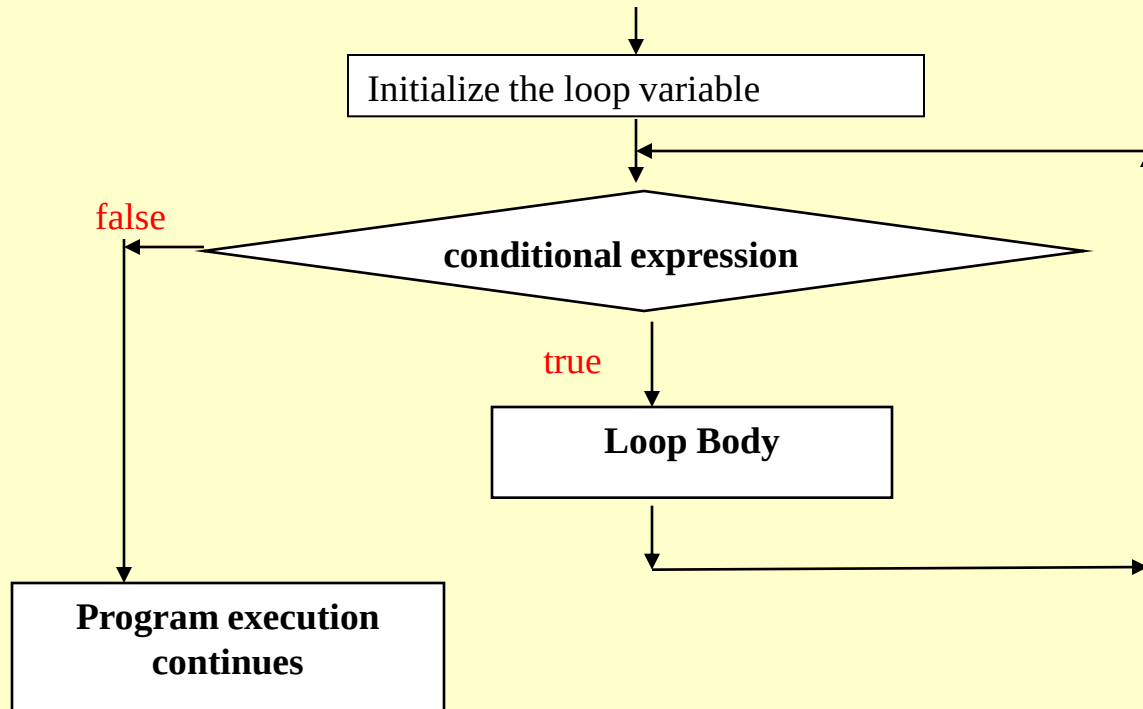
The *while* Statement

- The format of the *while* statement is as follows:

```
while ( conditional_expression )  
    body;
```

- Where - **while** is the Java keyword indicating a repetition.
- The conditional expression determines if the loop body must be executed
- The **while** statement specifies that the conditional expression must be tested at the beginning of the loop.
- If the conditional expression is initially false, then the loop body is skipped.

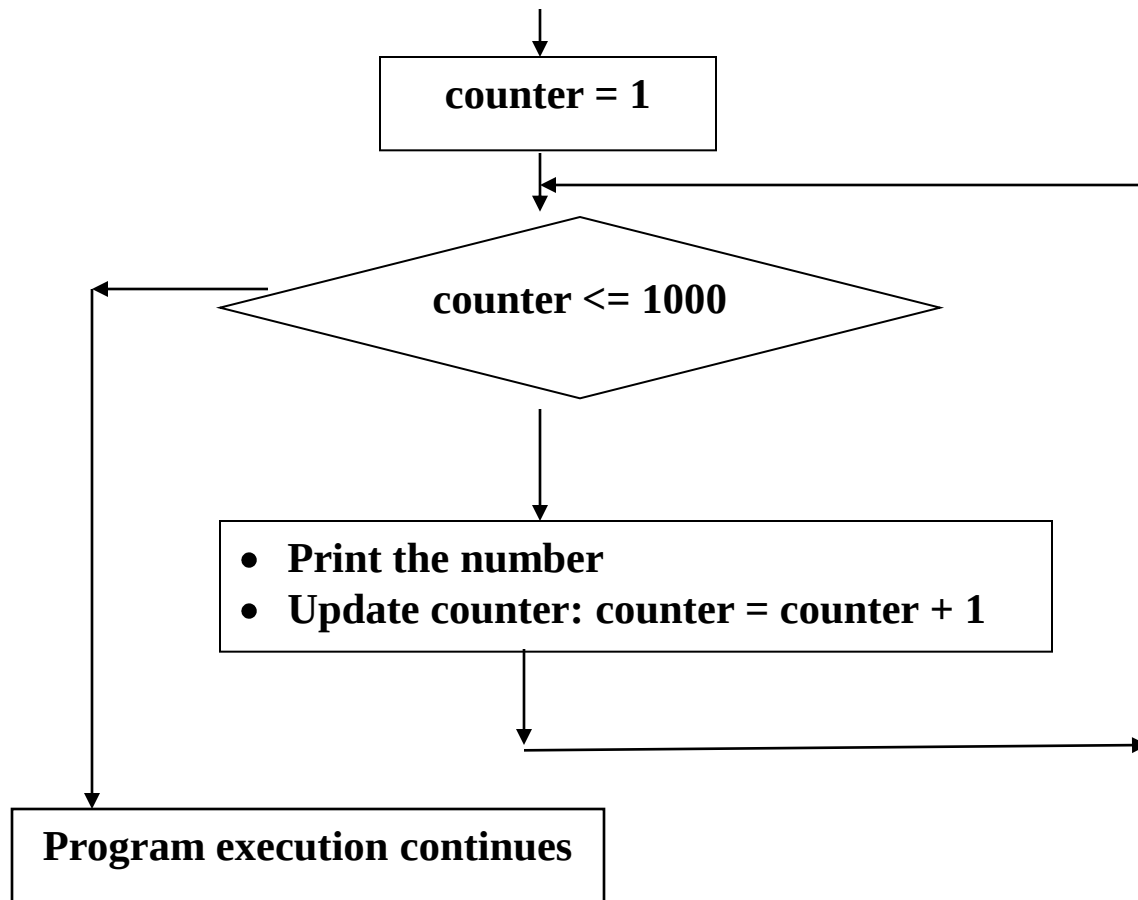
Iteration - symbolic representation of the *while* loop.



Iteration

- **Example 1**
Write a while loop that prints all the integers from 1 and 1000.
- **Solution**
- **The solution to this problem focuses on two key issues:**
 - 1. Count from 1 to 1000, and**
 - 2. Print the number.**

Iteration - while



Program code

```
1.  public class Integers
2.  {
3.      public static void main(String[] arg)
4.      {
5.          int counter = 1;
6.
7.          while (counter <=1000 )
8.          {
9.              System.out.println(counter);
10.             counter = counter + 1;
11.          }
12.      }
13. }
```

Example

- Design a class that accepts an integer value and prints the digits of the number in reverse order.
- For example, if the number is 234, the program should print 432, or if the number is 24500, it should print 00542.

Analysis:

1. Based on the problem definition, we cannot tell in advance the number of digits contained in a given number.
2. Focus is on printing the digits from the rightmost to the leftmost one.
3. That is, given a number, say, **N**, its rightmost digit is **$N \% 10$** .
4. For example, if **N** is **234**, then the first rightmost digit is $234 \% 10$, which is 4.
5. The next rightmost digit in **N** is determined by $N / 10$.
6. Using the example **234**, the next rightmost digit in **N** is $234 / 10$ which is **23**.
7. Steps 3 – 6 are repeated until the new value in **N** is zero.
 - For instance, let **N** = 234, then the process works this way:

N	Process	Digit printed
234	$234 \% 10$	4
23	$234 / 10$	
23	$23 \% 10$	3
2	$23 / 10$	
2	$2 \% 10$	2
0	$2 / 10$	

```
1.  public class ReverseNumber
2.  {
3.      private int n;
4.      private String s;
5.
6.      public ReverseNumber(int n)
7.      {
8.          this.n= n;
9.          s = "";
10.     }
11.
12.     void reverse()
13.     {
14.         while (n> 0)
15.         {
16.             s = s + n%10;
17.             n= n/10;
18.         }
19.     }
20.     public String toString()
21.     { return s; }
22. }
```

TestReverseNumber

```
1.  public class TestReverseNumber
2.  {
3.      public static void main(String[] arg)
4.      {
5.          ReverseNumber r = new ReverseNumber(24500);
6.          r.reverse();
7.          System.out.println(r);
8.      }
9.  }
```

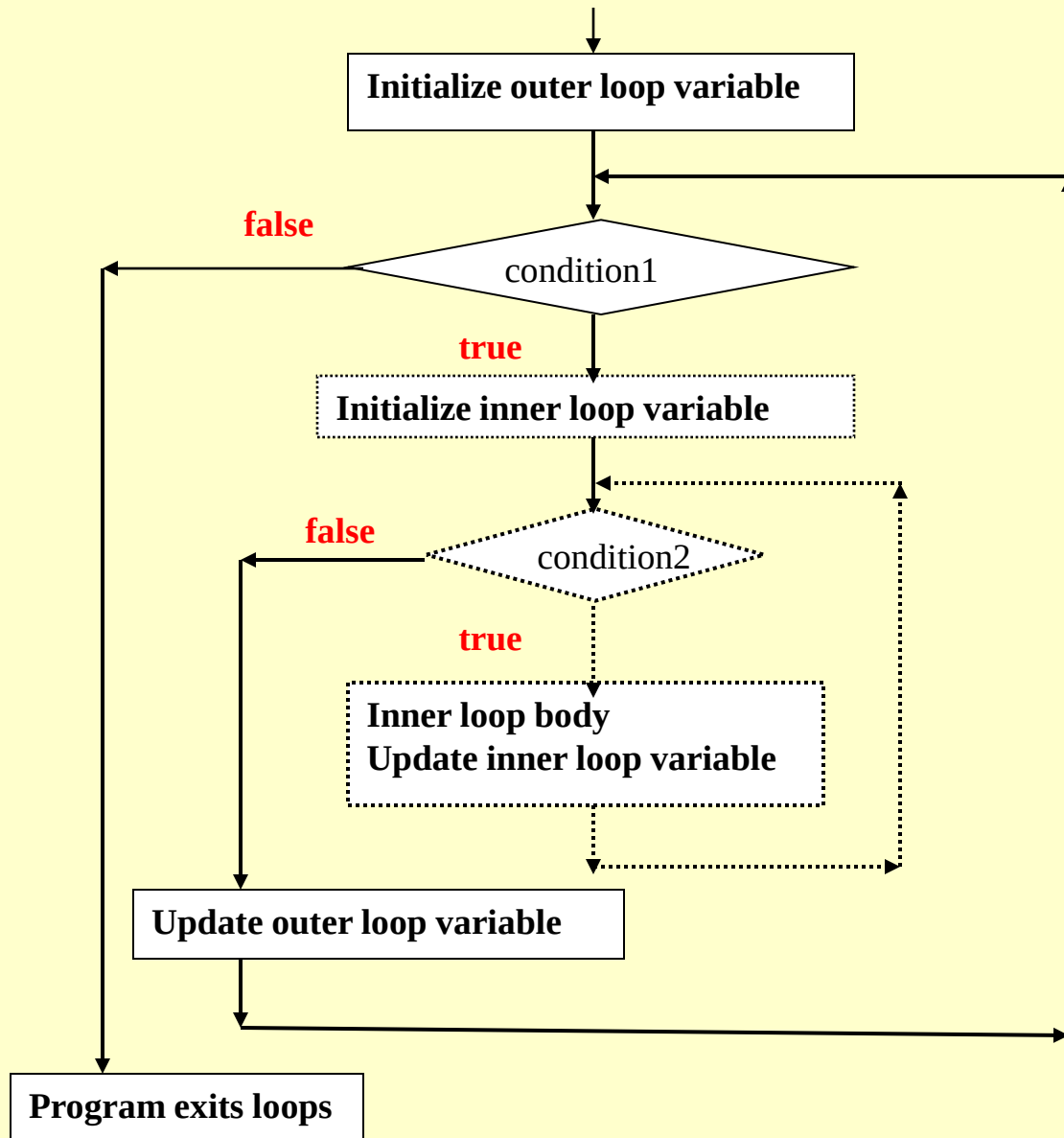
Nested while loop

- The **while** statement like the **if** statement, can be nested.
- Recall that the format of the **while** statement is:

```
while (condition)  
    S;
```

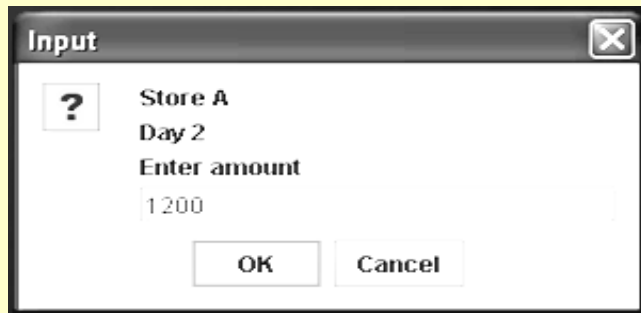
- Where S represents the loop body.
- If this is the case, then S itself can be a **while** statement.
- The construct would therefore be:

```
while (condition1)  
    while (condition2)  
        S;
```

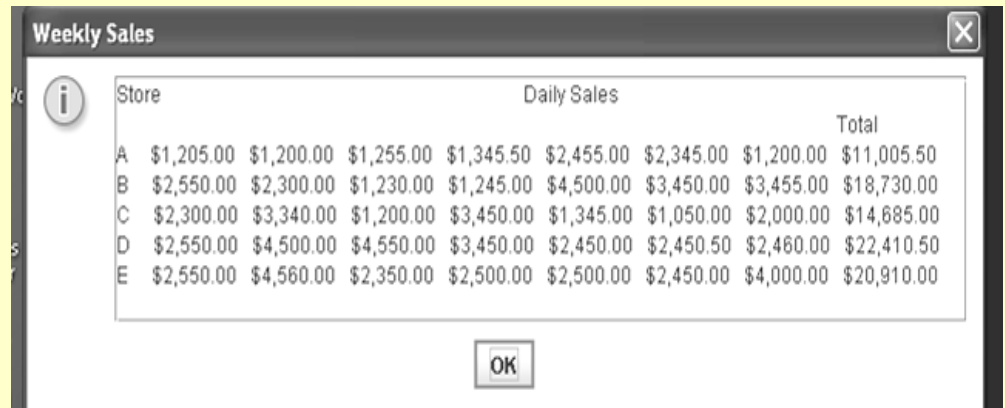


Nested while loop

- **Example 3**
- **A company has five stores – A, B, C, D, and E – at different locations across the state of Florida. At the end of each week the company's management prints a report of the daily sales and the total sales for each of the stores.**
- **Write a Java program that prints the sales and the total sales for each of the stores, row by row.**



A Java Swing dialog box titled "Input" with a close button (X) in the top right corner. It contains a question mark icon, the text "Store A", "Day 2", and "Enter amount". Below the text is a text input field containing the value "1200". At the bottom are "OK" and "Cancel" buttons.

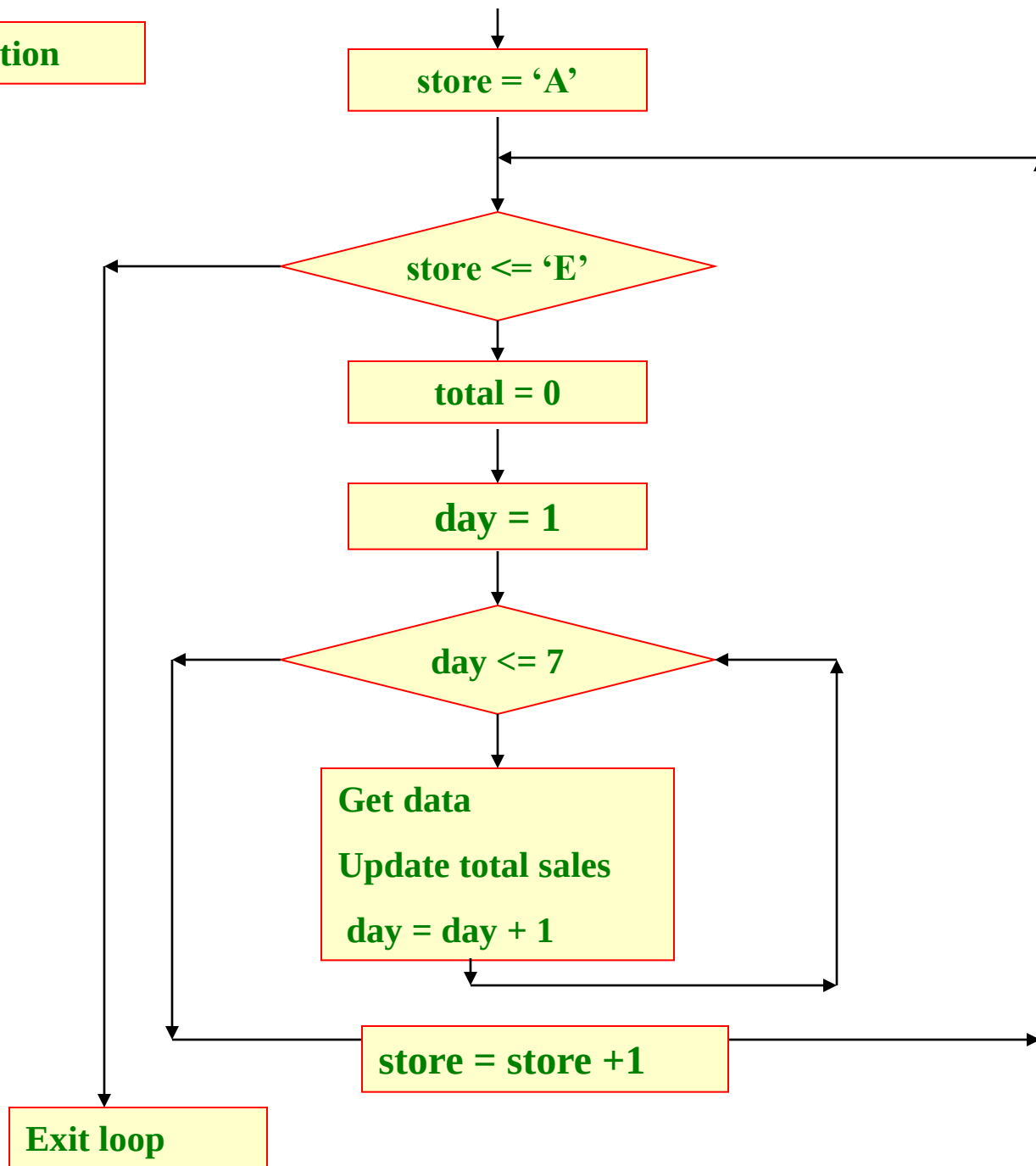


A Java Swing dialog box titled "Weekly Sales" with a close button (X) in the top right corner. It contains an information icon (i) and a table of sales data. The table has columns for "Store", "Daily Sales" (7 columns), and "Total". The data is as follows:

Store	Daily Sales							Total
A	\$1,205.00	\$1,200.00	\$1,255.00	\$1,345.50	\$2,455.00	\$2,345.00	\$1,200.00	\$11,005.50
B	\$2,550.00	\$2,300.00	\$1,230.00	\$1,245.00	\$4,500.00	\$3,450.00	\$3,455.00	\$18,730.00
C	\$2,300.00	\$3,340.00	\$1,200.00	\$3,450.00	\$1,345.00	\$1,050.00	\$2,000.00	\$14,685.00
D	\$2,550.00	\$4,500.00	\$4,550.00	\$3,450.00	\$2,450.00	\$2,450.50	\$2,460.00	\$22,410.50
E	\$2,550.00	\$4,560.00	\$2,350.00	\$2,500.00	\$2,500.00	\$2,450.00	\$4,000.00	\$20,910.00

An "OK" button is located at the bottom center of the dialog box.

Solution



```
1.  import javax.swing.JOptionPane;
2.  import java.text.NumberFormat;
3.  public class Sales
4.  {
5.      private double totalSales;
6.      private static final int DAYS = 7;
7.      private static final char STORES = 'E';
8.      private String header;
9.      private String s;
10.     private static final NumberFormat nf= NumberFormat.getCurrencyInstance();
11.     public Sales()
12.     {
13.         totalSales = 0;
14.         s = "";
15.         header = "Store\t\t\tDay\n\t1\t2\t3\t4\t5\t6\t7\tTotal\n";
16.     }
17.     public void calculateSales() { /*.....*/ }
38.     public String toString()
39.     {
40.         return header + s;
41.     }
42. }
```

```
17. public void calculateSales()
18. {
19.     char store = 'A';
20.
21.     while (store <= STORES) // Outer loop
22.     {
23.         s = s + store + " - ";
24.         int day = 1;
25.         totalSales = 0;
26.
27.         while (day <= DAYS) // Inner loop
28.         {
29.             double amount = GetData.getDouble("Store " + store + "\nDay " +
30.             day + "\nEnter amount"));
31.             s = s + "\t" + nf.format(amount);
32.             day++;
33.             totalSales = totalSales + amount;
34.         }
35.         s = s + "\t" + nf.format(totalSales) + "\n";
36.         store++;
37.     }
38. }
```

Class TestSales

```
1.  import javax.swing.JOptionPane;
2.  import javax.swing.JScrollPane;
3.  import javax.swing.JTextArea;
4.
5.  public class TestSales
6.  {
7.      public static void main(String[] arg)
8.      {
9.          Sales s = new Sales();
10.         s.calculateSales();
11.         JTextArea t = new JTextArea(s.toString(), 8, 50);
12.         JScrollPane p = new JScrollPane(t);
13.         JOptionPane.showMessageDialog(null, p, "Weekly Sales",
                                     JOptionPane.INFORMATION_MESSAGE );
14.     }
15. }
```

Iteration - while

Example 4

- The value of π can be determined by the series equation:

$$\pi = 4 (1 - 1/3 + 1/5 - 1/7 + 1/9 - 1/11 + 1/13 -)$$

- Write a class called PI that finds an approximation to the value of π , to 4 decimal places.
- Write a test class called TestPI that displays the value of π and the number of iterations used to find this approximation.

Analysis

Analysis

Each term in the series, beginning with the second is generated as follows:

$$x_1 = \frac{-1}{2*1+1} = \frac{-1}{3}$$

$$x_2 = \frac{1}{2*2+1} = \frac{1}{5}$$

$$x_3 = \frac{-1}{2*3+1} = \frac{-1}{7}$$

$$x_4 = \frac{1}{2*4+1} = \frac{1}{9}$$

:

:

In general the i^{th} term is generated by the following expression:

$$\frac{(-1)^i i}{2*i+1}$$

Generating new values

$$\frac{(-1)^i}{2 * i + 1}$$

**MARGIN OF
ERROR: 0.05**

i	x _{old}	x _{new}	abs(x _{new} - x _{old})
1	-0.3333	0.2000	0.5333333333333333
2	0.2000	-0.1429	0.34285714285714286
3	-0.1429	0.1111	0.25396825396825395
4	0.1111	-0.0909	0.20202020202020202
5	-0.0909	0.0769	0.16783216783216784
6	0.0769	-0.0667	0.14358974358974358
7	-0.0667	0.0588	0.12549019607843137
8	0.0588	-0.0526	0.11145510835913312
9	-0.0526	0.0476	0.10025062656641603
10	0.0476	-0.0435	0.09109730848861283

MARGIN_OF_ERROR = 0.05

- **i = 1**
- **x_{old} = (your initial guess)**
- **x_{new} = (generate new value using the expression)**

false

abs(x_{new} - x_{old}) > MOE

true

- **sum = sum + x_{new}**
- **i++**
- **x_{old} = x_{new}**
- **x_{new} = (generate new x value)**

Print result

Analysis

This type of problem requires you to:

- Compare the absolute value of the difference of two successive terms.
- If the value is less than the threshold or margin of error, then the last new value generated is the approximation to the actual value.
- In other words,

```
while ( |  $x_{\text{new}} - x_{\text{old}}$  | > some margin of error )  
{  
    sum the new value to previous amount  
    save the new as old.  
    generate the new value,  $x_{\text{new}}$   
}
```

Analysis

- In this example let the margin of error between two successive values be **0.05**.
- In addition, each new term may be generated as follows expression:

`Math.pow(-1.0, n)/(2 * n + 1);`

```
1.  import java.text.NumberFormat;
2.  import java.text.DecimalFormat;
3.  public class PI
4.  {
5.      static final double MARGIN_OF_ERROR = 0.05;
6.      double sum;
7.      int iterate;
8.
9.      public PI()
10.     {
11.         sum = 1;
12.     }
13.     public double findXnew(int n)
14.     {
15.         return Math.pow(-1.0, n)/(2 * n + 1);
16.     }
17.     public void findPi()
18.     {
19.         // Details to come.....
20.         while (Math.abs(xnew - xold) > MARGIN_OF_ERROR)
21.         {
22.             // To complete.....
23.         }
24.     }
25.     public String toString() { // .....}
26. }
```

```
1.  public void findPi()
2.  {
3.      int i = 1;
4.      double xold = 2.0; // Choose a value
5.      double xnew = findXnew(i);
6.
7.      while (Math.abs(xnew - xold) > MARGIN_OF_ERROR)
8.      {
9.          sum = sum + xnew;
10.         xold = xnew;
11.         i++;
12.         xnew = findXnew(i);
13.     }
14.
15.     iterate = i;
16. }
```

Re-defining the toString method

```
1.  public String toString()  
2.  {  
3.      NumberFormat nf = NumberFormat.getInstance();  
4.  
5.      DecimalFormat df = (DecimalFormat)nf;  
6.  
7.      df.applyPattern("0.00000000");  
8.  
9.      return "The approximate value of pi is " + df.format((4*sum)) + "\n"  
      + "The number of iterations is " + iterate;  
8.  }
```

Class TestPI

```
1.  public class TestPI
2.  {
3.      public static void main(String[] arg)
4.      {
5.          PI p = new PI();
6.          p.findPi();
7.          System.out.println(p);
8.      }
9.  }
```

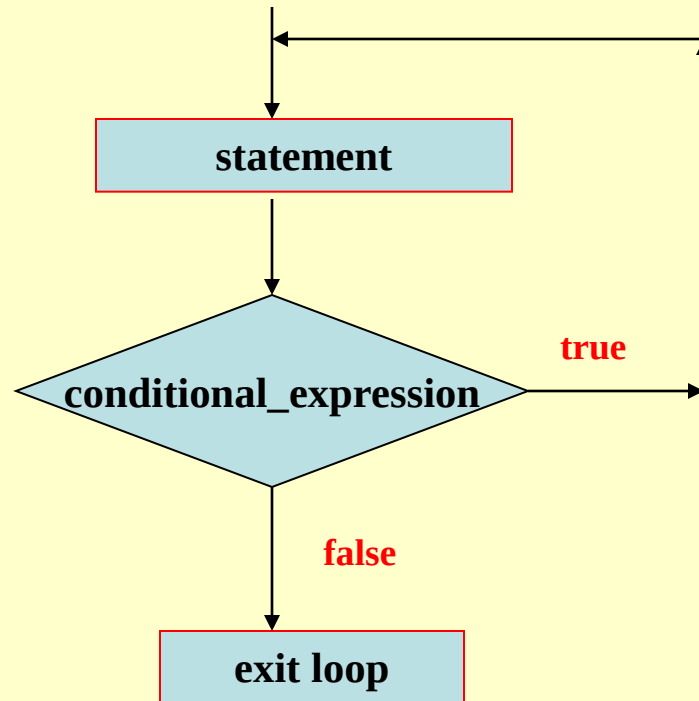
The *do...while* Statement

- The *do ...while* statement is in a way opposite to the *while* statement
- The conditional expression comes after the loop body.
- The format of the *do ... while* statement is:

```
do
{
    statement;
}
while (condition ) ;
```

- The word *do* is the keyword, indication the beginning of the loop.
- The pair of curly braces is mandatory.
- The statement finishes with the *while* clause.
- The conditional expression must be enclosed within parentheses.
- In addition, the entire statement must terminate with a semi-colon.

Diagrammatic view of the **do ... while** statement



Caution using the do/while

- The **do ...while** loop must be used with caution
- It attempts to execute the loop body without knowing if it is possible.
- For instance, consider the following segment of code:

```
int i = 1;  
do  
{  
    System.out.println( i/(i-1) );  
}  
while( i !=0 );
```

- Although the code is syntactically correct, it fails to execute.

The *for* Statement

- The *for* statement is the third form of looping construct.
- It behaves similar to the *while* and the *do...while* statements
- The general format of the *for* statement is as follows:

```
for ( data_type id = initialValue; conditional_expression; adjust_id )  
    statements;
```

The *for* loop is a single statement consisting of two major parts:

1. The loop heading, and
2. The loop body

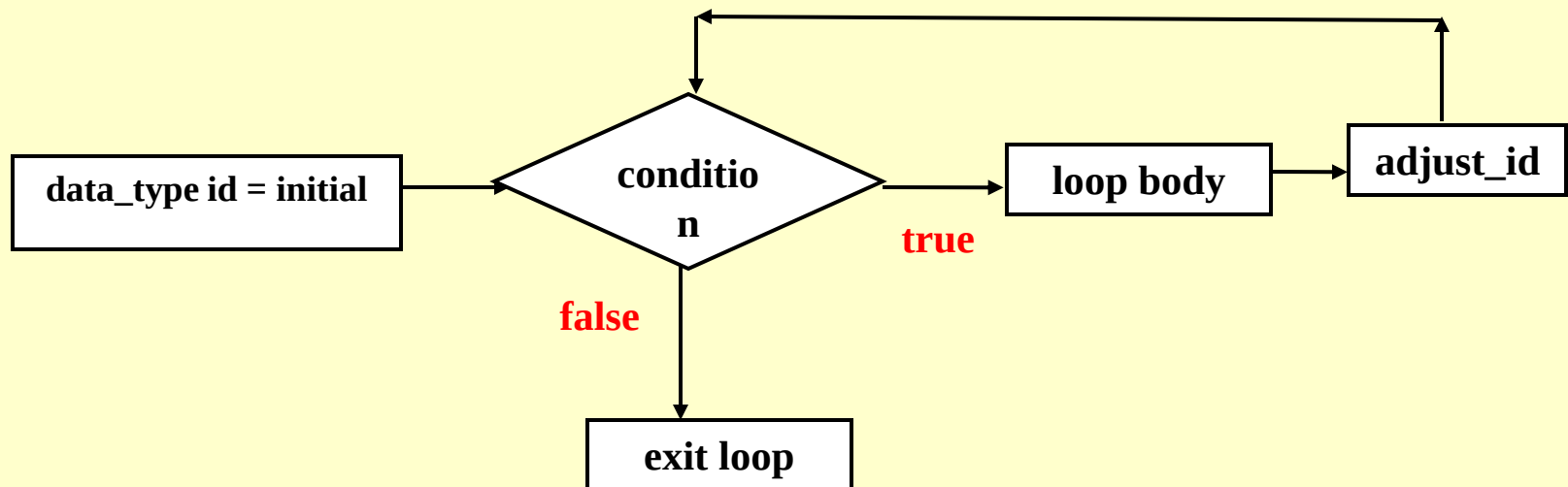
Loop Heading

The loop heading is enclosed within parentheses and consists of three expressions:

1. The first expression constitutes the declaration and initialization of the control variable for the loop. : **data_type id = initialValue**
2. The second expression constitutes the condition under which the loop iterates. **conditional_expression**
3. The third expression updates the value of the loop variable. **adjust_id**

The Loop Body

- The loop body constitutes a statement or a block of statements.
- It is executed only if the conditional expression is true, otherwise the loop terminates.



Behaviour of the **for** Loop

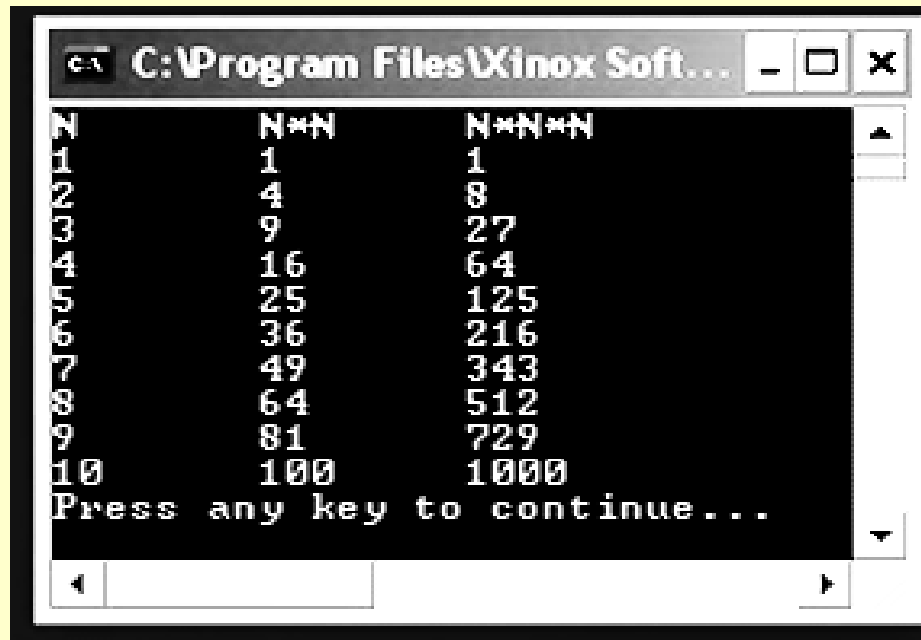
The **for** statement behaves the following way:

1. The control loop variable is declared and initialized.
2. The condition is tested. If the condition is true, the loop body is executed, if not, it terminates.
3. The loop variable is updated. That is, the control variable is re-assigned, and step 2 is repeated.

Using **for** Statement

Example:

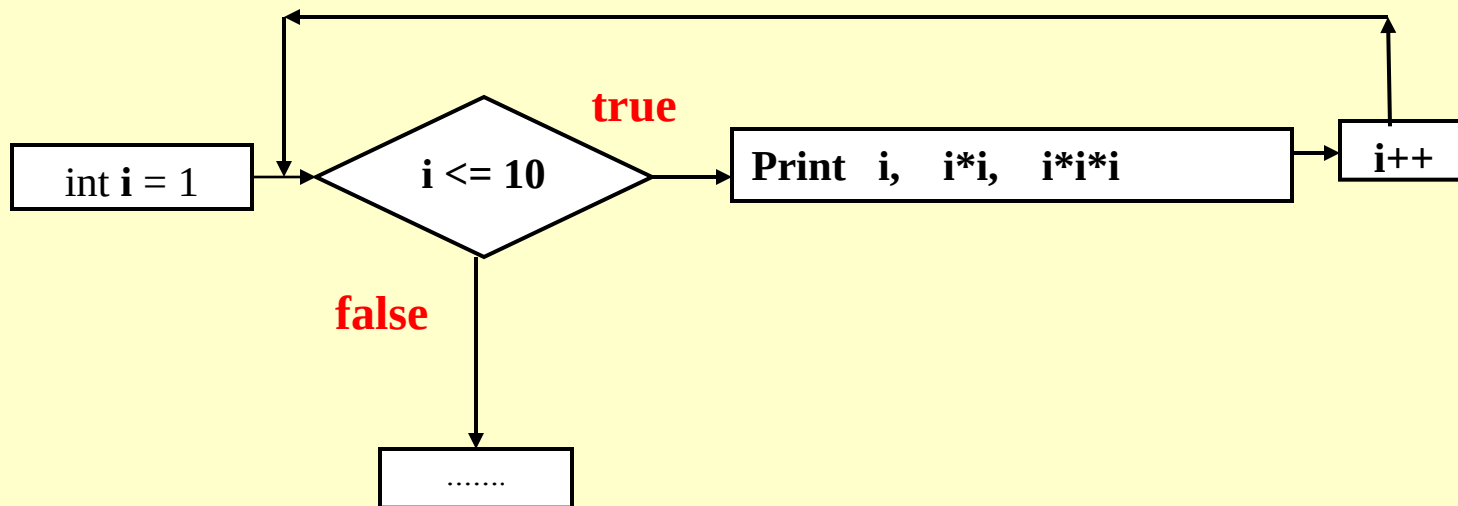
Write a program that lists the integers from 1 to 10, along with their squares and their cubes. That is, the program produces output:



```

N      N*N      N*N*N
1      1        1
2      4        8
3      9        27
4      16       64
5      25       125
6      36       216
7      49       343
8      64       512
9      81       729
10     100      1000
Press any key to continue...
```

Diagrammatic view of the Solution



Program Code

```
1.  public class Powers
2.  {
3.      public static void main(String[] arg)
4.      {
5.          System.out.println("N\tN*N\tN*N*N\n");
6.
7.          for ( int i = 1; i <= 10; i++ )
8.              System.out.println( i + "\t" + i*i + "\t" + i*i*i );
9.      }
10. }
```

Using **for** to Determine Palindrome

A palindrome is a set of data values that is identical to its reverse. For example the word **MADAM** is a palindrome; the number **123454321** is a palindrome; likewise **aaabbbaaaabbbaaa**.

Design a class called Palindrome that accepts a string and determines if the string is a palindrome.

To determine if a sequence of characters is a palindrome do the following:

- Compare the first and last character in the sequence. If they are the same,
- Compare the second character and the second to the last character in the sequence.
- Continue the process in similar manner. If all items match, then the string is a palindrome.
- Conversely, at the first unmatched occurrence, the string is not a palindrome.

Determine Palindrome

String: M A D A M

Index: 0 1 2 3 4



$\text{halfIndex} = \text{index}/2;$

Where $\text{index} = \text{str.length}()$

Compare characters on either side of half the index, beginning at opposite ends.

i.e. $\text{str.charAt}(i) == \text{str.charAt}(\text{index} - i - 1)$

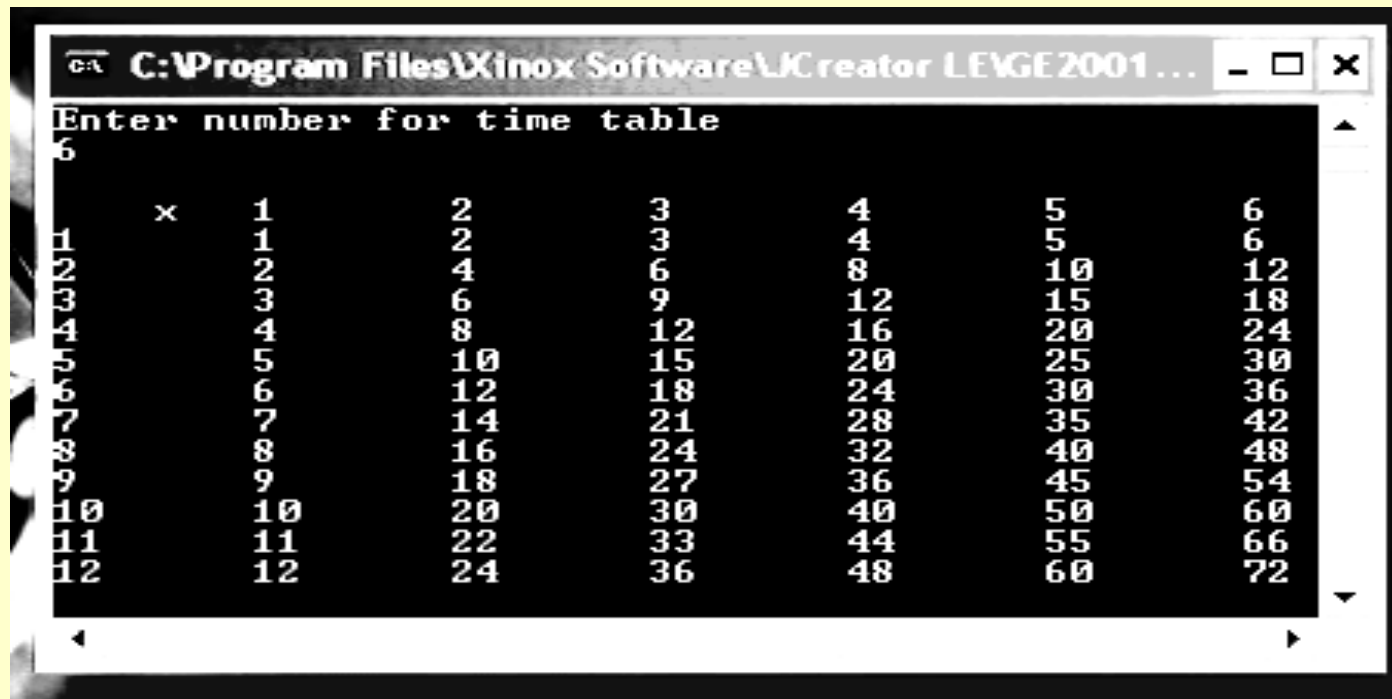
```
1.  public class Palindrome
2.  {
3.      private String word;
4.      private boolean palindrome;
5.      private int index, halfIndex;
6.
7.      public Palindrome(String s)
8.      {
9.          word = s;
10.         index = s.length();
11.         palindrome = true;
12.         halfIndex = index/2;
13.     }
14.
15.     boolean isPalindrome()
16.     {
17.         for ( int i = 0;  i < halfIndex && palindrome;  i++ )
18.             if ( word.charAt(i) != word.charAt(index - i - 1) )
19.                 palindrome = false;
20.         return palindrome;
21.     }
22. }
```

TestPalindrome

```
1.  public class TestPalindrome
2.  {
3.      public static void main(String[] arg)
4.      {
5.          Palindrome p = new Palindrome("aaabbbaaaabbaaa");
6.          System.out.println(p.isPalindrome());
7.      }
8.  }
```

A Time Table

Design a class that prints any timetable.



```
C:\Program Files\Xinox Software\UCreator LEVGE2001...
Enter number for time table
6
  ×   1   2   3   4   5   6
1  1   2   3   4   5   6
2  2   4   6   8  10  12
3  3   6   9  12  15  18
4  4   8  12  16  20  24
5  5  10  15  20  25  30
6  6  12  18  24  30  36
7  7  14  21  28  35  42
8  8  16  24  32  40  48
9  9  18  27  36  45  54
10 10 20 30 40 50 60
11 11 22 33 44 55 66
12 12 24 36 48 60 72
```

```
1. public class TimeTable
2. {
3.     int n;
4.     String s ;
5.
6.     Timetable(int n)
7.     {
8.         this.n = n;
9.         s = "\n  x";
10.    }
11.
12.    void makeTimeTable()
13.    {
14.        for (int i = 1; i <= n; i++)
15.            s = s + "\t" + i;
16.
17.        s = s + "\n";
18.
19.        for (int row= 1; row<= 12; row++)
20.        {
21.            s = s + row;
22.            for (int col= 1; col<= n; col++)
23.                s = s + "\t" + (row * col);
24.            s = s + "\n";
25.        }
26.    }
```

```
27.     public String toString()
28.     {
29.         return s;
30.     }
31. }
```

```
1.  import java.util.Scanner;
2.
3.  public class NestedFor
4.  {
5.      public static void main(String[] arg)
6.      {
7.          System.out.println("Enter number for time table");
8.          Scanner read = new Scanner(System.in);
9.          int n = read.nextInt();
10.
11.          Timetable t = new TimeTable( n );
12.
13.          t.makeTable();
14.
15.          System.out.println(t);
16.      }
17.  }
```